

로그 구조 기반 RCA 와 KV Cache 압축 파이프라인

로그 파싱 · 장애 구간 추출 · 분할 · 특징 추출 · LLM 입력 · 증거 보존형 KV 압축

목적

문서의 목적

자동 로그 분석 서버에서 설명한 전처리·분석 흐름을 정리하고, 이를 긴 로그 RCA 용 LLM 입력 및 로그 구조 기반 KV Cache 압축으로 확장하는 연구 가설을 하나의 문서로 연결한다.

정리 기준

A Survey on Automated Log Analysis for Reliability Engineering [1]
및 대화에서 설계한 로그 RCA 특화 KV Cache 압축 아이디어

2026.07.20

목차

- 1. 전체 핵심과 연구 문제
- 2. 자동 로그 분석의 전체 파이프라인
- 3. 로그 파싱: 템플릿과 파라미터 분리
- 4. 장애 구간 추출과 로그 분할
- 5. 특징 추출: 시퀀스·카운트·파라미터·그래프
- 6. LLM 입력 표현 설계
- 7. 로그 구조 기반 KV Cache 압축
- 8. 기존 방식보다 좋아질 가능성과 위험
- 9. 실험 설계와 비교 기준
- 10. 최종 권장 파이프라인과 구현 체크리스트
- 11. 핵심 용어와 참고문헌

요약	먼저 기억할 한 문장 파싱은 한 줄의 구조를 찾는 작업이고, 장애 구간 추출은 관련 로그의 범위를 찾는 작업이며, 특징 추출은 LLM 이 읽을 표현을 만드는 작업이다. KV Cache 압축은 이 구조를 이용해 반복 KV 는 줄이고 RCA 증거 KV 는 보존하는 단계다.
----	---

1. 전체 핵심과 연구 문제

1.1 서베이 논문이 말하는 핵심

이 서베이 논문은 새로운 파서나 진단 모델 하나를 제안한 것이 아니라, 자동 로그 분석 연구를 로그 생성부터 압축·파싱·마이닝·진단까지 하나의 생애주기로 정리한다. 로그는 운영 중 시스템의 실제 상태와 실행 흐름을 기록하는 핵심 증거이지만, 현대 시스템에서는 양이 매우 크고 형식이 다양하며 여러 요청과 서비스가 뒤섞인다. 따라서 사람이 규칙을 일일이 작성하는 방식만으로는 확장하기 어렵고, 전체 과정을 자동화해야 한다는 것이 논문의 출발점이다. [1]

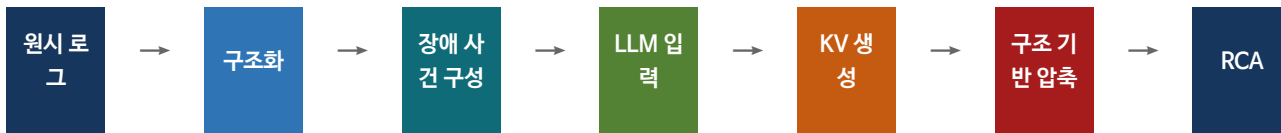
관점

가장 중요한 관점

로그 분석의 성능은 마지막 AI 모델만으로 결정되지 않는다. 필요한 로그를 남기지 않거나, 파싱·분할·압축 과정에서 증거를 잃으면 뒤의 모델이 아무리 좋아도 근본 원인을 찾을 수 없다.

1.2 현재 연구로 확장한 질문

현재 연구는 서베이의 전체 흐름을 LLM 추론 과정까지 확장한다. 핵심 질문은 “긴 로그를 LLM에 입력할 때, 로그 구조를 이용해 어떤 KV를 보존하고 어떤 KV를 제거해야 RCA 정확도를 유지할 수 있는가?”이다.



가설

핵심 가설

로그의 템플릿·파라미터·반복·시간 순서·식별자·서비스 연결성을 KV 보존 정책에 직접 이용하면, 일반적인 최근성 또는 attention 중심 KV 압축보다 같은 메모리 예산에서 원인 증거를 더 잘 보존할 가능성이 있다.

1.3 성공을 정의하는 기준

- Full KV Cache 보다 항상 정확도가 높다는 주장은 현실적인 목표가 아니다.
- 같은 KV 예산에서 더 높은 RCA 정확도와 원인 증거 보존율을 얻는 것이 핵심이다.
- 같은 RCA 정확도에서 더 적은 KV 메모리와 더 낮은 디코딩 비용을 달성하는 것도 성공이다.
- 전처리와 압축의 오버헤드를 포함한 전체 실행 시간이 개선되어야 한다.

2. 자동 로그 분석의 전체 파이프라인



논문은 자동 로그 분석을 Logging, Log Compression, Log Parsing, Log Mining 으로 정리한다. Log Mining 안에는 로그 분할, 특징 추출, 모델 학습, 이상 탐지·실패 예측·실패 진단이 포함된다. [1]

단계	핵심 질문	잘못되었을 때 생기는 문제
Logging	어디에, 무엇을, 어떤 형식으로 기록할 것인가	원인 상태 자체가 기록되지 않아 복구 불가능
Compression	반복을 얼마나 줄이고 어떤 정보를 남길 것인가	원인·중간 증거가 삭제됨
Parsing	고정 템플릿과 실행 파라미터를 어떻게 나눌 것인가	이벤트 종류와 상태값이 왜곡됨
Partitioning	어떤 로그들을 하나의 요청·장애 사례로 볼 것인가	서로 다른 요청이 섞이거나 원인과 결과가 갈림
Feature extraction	순서·빈도·값·관계 중 무엇을 표현할 것인가	모델이 필요한 단서를 보지 못함
RCA	결과 로그가 아니라 근본 원인을 어떻게 찾을 것인가	ERROR 문장만 반환하고 원인 설명 실패

원칙	연구 설계 원칙 각 단계는 독립적인 전처리가 아니라 다음 단계가 볼 수 있는 증거의 범위를 결정한다. 따라서 파서 정확도, 장애 구간 포함률, 특징 표현, KV 보존율을 함께 평가해야 한다.
----	--

3. 로그 파싱: 템플릿과 파라미터 분리

3.1 파싱의 정의

로그 파싱은 반구조적인 원시 로그 한 줄을 정형 필드, 이벤트 템플릿, 파라미터로 분해하는 작업이다. 파서가 장애 구간을 직접 찾는 것은 아니다.

원시 로그 2026-07-15 10:03:21 ERROR node-3 DB connection timeout host=10.0.0.5
--

필드	추출값	의미
timestamp	2026-07-15 10:03:21	사건 발생 시점
level	ERROR	로그 심각도
component	node-3	로그를 생성한 실행 주체
template	DB connection timeout host=(<*>)	반복되는 사건의 종류
parameter	10.0.0.5	실행 당시의 실제 상태값

구분	구분 템플릿은 “어떤 사건인가”를 나타내고, 파라미터는 “그 사건에서 실제 상태가 무엇이었는데가”를 나
----	---

타낸다. RCA 에서는 둘 다 필요하다.

3.2 파싱 오류가 전체 파이프라인에 미치는 영향

오류 유형	예시	후속 영향
과도한 병합	status=200 과 status=500 을 같은 사건으로만 취급	정상과 장애가 구분되지 않음
과도한 분리	같은 사건을 값마다 다른 템플릿으로 생성	이벤트 종류가 폭증하고 카운트가 분산됨
파라미터 유실	timeout=30 과 timeout=0 을 모두 TimeoutConfigured 로만 표현	상태 변화 원인을 잃음
식별자 유실	request_id, block_id 를 일반 숫자로 마스킹	요청·작업 단위 분할이 불가능해짐

3.3 현재 실용적인 파서 선택

새로 개발하는 서비스는 JSON·Key-Value 같은 구조화 로그를 우선하고, 이미 형식이 정해진 로그는 Regex·Grok 류 규칙을 사용하며, 형식을 모르는 대규모 반구조 로그에는 Drain3 같은 온라인 템플릿 파서를 기준선으로 둘 수 있다. OpenTelemetry 로그 데이터 모델은 Timestamp, TraceId, SpanId, Severity, Body, Attributes, EventName 등의 공통 필드를 정의한다. [2] Drain3 는 고정 길이 파스 트리를 이용해 스트리밍 로그에서 템플릿을 온라인으로 학습하고 파라미터를 추출한다. [3]

상황	권장 접근	이유
직접 제어하는 신규 서비스	구조화 JSON / OpenTelemetry	파싱 추측을 최소화하고 trace 연계를 쉽게 함
형식이 고정된 서버 로그	Regex·Grok·Dissect	필드 위치와 구분자가 명확함
대량의 반구조 로그	Drain3	빠른 온라인 템플릿 학습과 상태 저장 가능
새로운 형식이 자주 등장	Drain3 + 선택적 LLM 보정	기본 파서는 속도, LLM 은 애매한 케이스 보완

권장 **프로젝트 권장안**
timestamp·level·component·식별자는 규칙으로 먼저 추출하고, 메시지 본문은 Drain3 로 템플릿화하며, 템플릿 ID 와 원본 파라미터를 모두 저장한다.

4. 장애 구간 추출과 로그 분할

4.1 파싱과 장애 구간 추출은 다른 작업

작업	입력 단위	출력	핵심 질문
로그 파싱	로그 한 줄	필드·템플릿·파라미터	이 문장은 어떤 사건이고 실제 값은 무엇인가?
장애 구간 추출	전체 로그 스트림	하나의 장애 사례에 해당하는 범위	어느 시점부터 어느 시점까지가 같은 장애인가?
로그 분할	추출된 범위 또는 전체 로그	요청·작업·서비스별 그룹	어떤 로그들이 같은 실행 단위 에 속하는가?

권장 순서

기본 파싱 → 장애 anchor 결정 → 장애 구간 추출 → 정밀 템플릿 파싱 → 특징 추출

4.2 장애 구간의 기준점(anchor)

- 데이터셋의 이상·장애 라벨이 붙은 로그 시점
- 실패한 request_id·job_id·block_id의 종료 시점
- 서비스 중단·HTTP 실패·프로세스 종료 등 명시적 실패 이벤트
- 장애 티켓이나 운영 기록에 표시된 발생 시각

anchor는 최종 실패의 위치일 뿐, 반드시 근본 원인 로그는 아니다. 원인은 anchor보다 훨씬 앞에 존재할 수 있으므로 전후 문맥을 포함해야 한다.

4.3 고정 윈도우와 슬라이딩 윈도우

시간 기반 분할

고정 윈도우: 10:00-10:10 | 10:10-10:20
슬라이딩 윈도우: 10:00-10:10
10:02-10:12
10:04-10:14

항목	고정 윈도우	슬라이딩 윈도우
경계 문제	원인과 결과가 경계 양쪽으로 갈릴 수 있음	겹치는 구간에서 함께 포함될 가능성이 높음
시작점 민감도	시작 시각에 따라 샘플 구성이 크게 달라짐	여러 시작점에서 반복 관찰하여 민감도가 낮음
온라인 갱신	윈도우가 끝나야 새 결과 생성	짧은 step마다 최근 문맥을 재평가 가능
비용·중복	낮음	같은 로그가 여러 샘플에 중복되어 비용 증가
데이터 누수	비교적 관리하기 쉬움	겹친 윈도우를 무작위 분할하면 동일 장애가 train/test에 섞일 수 있음

장점

슬라이딩 윈도우의 핵심 장점

장애 직전의 원인 로그와 장애 직후의 결과 로그가 고정 경계 때문에 분리되는 문제를 줄이고, 장애 전조·발생·전파를 서로 다른 문맥에서 관찰할 수 있다.

4.4 식별자 기반 분할

식별자 기반 분할은 “비슷한 시간에 발생한 로그”가 아니라 “같은 실행 작업에 속한 로그”를 묶는 방식이다. 논문은 HDFS 의 block_id 처럼 작업 대상을 식별하는 값을 이용해 관련 로그를 그룹화하는 방식을 설명한다. [1]

식별자	묶는 범위	주의점
trace_id	여러 서비스를 통과하는 하나의 요청 전체	중간 서비스에서 전파되지 않으면 연결이 끊김
span_id	요청 내부의 특정 서비스·함수 작업	부모-자식 관계가 있어야 호출 그래프 구성 가능
request_id / session_id	애플리케이션 요청 또는 사용자 세션	서비스마다 필드명이 다를 수 있음
job_id / task_id	배치·분산 작업	재시도나 하위 작업 ID 와의 관계 확인 필요
block_id	스토리지 데이터 블록 생애주기	동일 블록의 장기 이벤트가 너무 넓게 묶일 수 있음
process_id / thread_id	실행 프로세스·스레드	스레드 재사용 때문에 여러 요청이 합쳐질 수 있음
host / pod / component	발생 위치·실행 주체	요청 단위보다 범위가 넓어 시간 조건과 결합 필요

현실적인 하이브리드 분할

1 순위: trace_id / request_id / job_id로 묶기

2 순위: 같은 ID 라도 긴 시간 간격이 생기면 다시 나누기

3 순위: ID 가 없으면 service · host · component + 시간 윈도우

4 순위: 여러 ID 와 서비스 관계를 그래프로 연결하기

주의

데이터셋 분할 주의

전체 로그에서 겹치는 윈도우를 만든 뒤 무작위로 train/test 를 나누면 데이터 누수가 발생할 수 있다. 먼저 장애 사건·날짜·시스템 단위로 분리하고, 각 분할 내부에서 윈도우를 만드는 편이 안전하다.

5. 특징 추출: 시퀀스·카운트·파라미터·그래프

5.1 이벤트 시퀀스

각 템플릿에 이벤트 ID 를 부여하고 발생 순서대로 나열한다. 시간 순서 자체가 특징인 이유는 프로그램이 상태를 순차적으로 바꾸며 실행되기 때문이다. 같은 이벤트 집합이라도 순서가 다르면 정상 제어 흐름과 장애 흐름이 달라진다.

같은 카운트, 다른 의미

정상: Start → Validate → Commit

이상: Start → Commit → Validate

- 필요한 단계의 누락, 예상하지 못한 이벤트 삽입, 순서 역전, 반복 루프를 표현한다.
- ConfigChanged → Timeout 과 Timeout → ConfigChanged 는 원인 후보와 사후 조치로 의미가 달라질 수 있다.
- 순서뿐 아니라 이벤트 사이 시간 간격 Δt 를 함께 넣으면 지연과 정체를 표현할 수 있다.
- 분산 시스템에서는 서버 시계 차이가 있으므로 trace/span 부모-자식 관계와 로컬 순서를 함께 사용해야 한다.

5.2 이벤트 카운트

하나의 윈도우 또는 장애 사건에서 각 이벤트가 몇 번 발생했는지를 벡터로 표현한다. 순서를 잃는 대신 재시도·타임아웃·경고의 폭증을 잘 나타낸다.

카운트 특징

Retry=3,000 | Timeout=20 | RequestFailed=10

5.3 파라미터 특징

같은 템플릿 안에서 실제 값의 상태와 변화를 보존한다. timeout=30→1, memory_limit=16GB→512MB 처럼 템플릿은 같지만 값 변화가 근본 원인인 장애를 찾는 데 필요하다.

5.4 그래프형 특징

그래프는 로그를 한 줄로만 나열하지 않고 이벤트·서비스·요청·리소스를 노드로, 호출·전이·동일 식별자·시간 관계를 간선으로 표현한다. 논문은 실행 순서, 의존성, 동시 발생, 계층 관계를 그래프 형태로 추출하는 연구들을 정리한다. [1]

그래프 종류	노드	간선	적합한 분석
이벤트 전이 그래프	이벤트 템플릿	Ei 다음에 Ej 가 발생	정상 전이 위반, 반복 루프
서비스 호출 그래프	서비스·컴포넌트	호출·의존성	장애 전파 경로
로그 인스턴스 그래프	실제 로그 한 줄	시간·ID·같은 서비스 관계	구체적 증거 chain 추적
이종 그래프	로그·서비스·요청·DB·호스트	소속·호출·사용 리소스	멀티홉 RCA 와 복합 원인

이종 그래프 예시

```

Trace T1
├── Gateway 로그
├── Order 로그
├── Payment 로그
│   └── Database
│       └── timeout 로그

```

균형

그래프의 장점과 한계

시퀀스가 표현하기 어려운 병렬 분기·합류·서비스 전파를 표현할 수 있다. 그러나 A 다음에 B 가 자주 나타난다는 통계만으로 A 가 B 의 실제 원인이라고 단정하면 안 된다.

5.5 시퀀스와 카운트를 섞는 방법

방법	표현	장점
이중 인코더	시퀀스→Transformer, 카운트→MLP, 두 표현 결합	순서와 전체 빈도를 명시적으로 분리해 학습
이벤트별 부가 특징	각 이벤트 토큰에 window_count, Δt, level 추가	기존 시퀀스 모델에 쉽게 결합
N-gram 카운트	Ei→Ej 전이 횟수 행렬	순서와 빈도를 동시에 보존하는 간단한 그래프
Run-Length Encoding	(Retry, count=3000)처럼 연속 반복을 하나로 압축	순서·반복량을 유지하면서 입력을 크게 줄임
지역 시퀀스 + 전체 통계	장애 주변은 상세 순서, 긴 구간은 카운트 요약	원인 흐름과 전체 폭증을 동시에 제공

권장 결합 표현

```
(Connect, count=1)
→ (Retry, count=3000, duration=30s)
→ (Timeout, count=1)
→ (RequestFailed, count=1)
```

예외

중요한 예외

동일 템플릿이라도 주요 파라미터가 달라지면 하나로 합치면 안 된다. Retry(host=A, timeout) × 2 와 Retry(host=B, permission) × 1 은 상태 변화 지점에서 분리해야 한다.

6. LLM 입력 표현 설계

6.1 원시 로그를 그대로 넣는 방식의 문제

- Heartbeat·Health check·Retry 같은 반복이 입력 대부분을 차지한다.
- 서로 다른 요청의 로그가 시간순으로 뒤섞이면 LLM 이 존재하지 않는 인과관계를 만들 수 있다.
- 희귀한 원인 INFO 로그가 긴 문맥 중간에 묻힐 수 있다.
- 입력 토큰 수, prefill 연산량, 최초 KV Cache 크기가 함께 증가한다.

6.2 구조화 입력의 권장 단위

LLM 입력의 기본 단위를 단순 토큰이나 원시 한 줄이 아니라 “연속된 동일 상태의 로그 구간”으로 잡는 것이 유용하다.

구조화 chunk 메타데이터

```
{
  template_id,
  repeat_count,
  first_timestamp, last_timestamp,
```

```
service, level, identifier,
parameter_state, parameter_change,
predecessor / successor
}
```

6.3 LLM 에 제공할 계층적 문맥

층	포함 정보	역할
사건 요약	장애 시각, 영향 서비스, 전체 카운트	전체 범위와 심각도 파악
증거 시퀀스	원인 후보→중간 증상→결과	RCA 의 핵심 논리 구성
파라미터 변화	설정·자원·상태값의 before/after	같은 템플릿 내부의 원인 발견
식별자·그래프	trace/request/job 및 서비스 연결	다른 요청 혼입 방지, 멀티홉 추론
대표 원문	각 중요 chunk 의 실제 로그 일부	구조화 과정에서 빠진 세부 의미 보완

차이	입력 압축과 KV 압축의 차이 입력 구조화는 LLM 이 읽기 전에 토큰과 prefill 비용을 줄인다. KV Cache 압축은 LLM 이 읽은 뒤 생성된 내부 상태를 줄여 decode 메모리와 attention 비용을 낮춘다. 두 단계는 서로 다른 병목을 해결한다.
----	---

7. 로그 구조 기반 KV Cache 압축

7.1 기존 KV 압축의 대표 기준

H2O 는 누적 attention 이 큰 heavy-hitter 토큰과 최근 토큰을 균형 있게 보존하고, SnapKV 는 프롬프트 끝 observation window 의 head 별 attention 패턴으로 중요한 KV 위치를 선택한다. [4] [5] ChunkKV 는 개별 토큰 대신 의미 chunk 를 압축 단위로 사용하여 토큰 간 의존성을 보존하려 한다. [6]

방식	주요 신호	로그 RCA 에서의 잠재 한계
Recent / sliding cache	최근 위치	초반 또는 중간의 원인 로그 유실
H2O	누적 attention + 최근성	많이 주목받는 결과 ERROR 와 희귀 원인을 구분하지 못할 수 있음
SnapKV	관찰 구간의 head 별 중요 위치	질문 직전 attention 이 초반 상태 변화까지 안정적으로 포착하지 못할 수 있음
ChunkKV	의미 chunk 중요도	일반 언어 chunk 와 로그의 템플릿·상태 구간은 구조가 다름
Layer / head-aware	계층·head 별 예산	로그 도메인의 원인·전파 역할을 직접 사용하지 않음

7.2 제안 방식의 압축 단위

차별점은 로그 구조를 LLM 입력 생성에만 쓰지 않고, KV 위치의 보존·제거 단위에 직접 연결하는 것이다. 각 로그 line 또는 구조화 chunk 와 해당 토큰 범위를 연결한 뒤, 동일한 템플릿·파라미터 상태·요청 범위에서 반복되는 KV 는 대표화한다.

단순 template 중복보다 강한 조건

압축 단위 = 같은 template
 + 같은 주요 parameter 상태
 + 같은 request/service 범위
 + 같은 연속 구간

7.3 우선 보존할 KV

보존 대상	이유
파라미터 상태가 바뀌는 경계	같은 템플릿 내부의 실제 원인일 수 있음
정상 실행에는 없던 희귀 이벤트	빈도는 낮지만 장애 시작점일 수 있음
식별자·서비스가 바뀌는 경계	요청과 서비스 전파 구조를 유지함
원인 후보→중간 증상→결과 chain	RCA 설명과 검증에 필요한 연결 증거
질문과 최종 장애 결과	생성 목표와 현재 문제를 명확하게 유지함
각 반복 구간의 대표 로그	반복량을 줄이면서 사건 의미를 남김

7.4 강하게 압축할 KV

- 같은 템플릿·같은 파라미터 상태·같은 서비스에서 연속 반복되는 Heartbeat·Polling·Health check
- 이미 대표 chunk 와 카운트로 요약된 동일 상태 재시도
- 원인 chain 과 연결되지 않고 다른 요청에 속한 정상 로그
- 형식 유지에만 필요한 상수 토큰 중 여러 chunk 에서 반복되는 부분

정책

핵심 정책

반복 횟수는 메타데이터로 남기고 동일 상태의 반복 KV 는 대표화하되, 템플릿·파라미터·서비스·식별자·전이 관계가 변하는 경계의 KV 는 우선 보존한다.

7.5 Head-aware 확장

모든 attention head 가 같은 역할을 한다고 가정하지 않고, 실제 attention 과 ablation 을 관찰해 head 별 행동을 사후적으로 분류할 수 있다. 2026 년 CompressKV 도 의미 검색에 특화된 head 를 식별해 KV 선택에 활용하는 방향을 제안한다. [7]

관찰 가능한 head 성격	압축 정책 후보
원인·중간 증거를 검색하는 head	관련 로그 chunk 에 더 큰 예산 배정
최근 결과와 질문을 보는 head	최근 window 와 실패 결과 보존

관찰 가능한 head 성격	압축 정책 후보
지역 형식·구분자를 보는 head	반복 템플릿 구간을 더 강하게 압축
파라미터 상태를 추적하는 head	상태 변화 전후 KV 를 보호
반복 로그에만 과도하게 집중하는 head	반복 대표화 정책과 함께 ablation 검증

검증

주의
head 역할은 이름을 먼저 정해 놓고 맞추면 안 된다. 특정 head 의 KV 를 제거하거나 chunk 를 가
린 뒤 RCA 답변과 원인 증거 회수가 어떻게 변하는지 실험으로 확인해야 한다.

8. 기존 방식보다 좋아질 가능성과 위험

8.1 좋아질 가능성이 큰 이유

- 로그의 반복은 일반 자연어보다 template·parameter·identifier 로 명확하게 정의할 수 있다.
- 입력 구조화가 prefill 과 최초 KV 크기를 줄이고, KV 압축이 decode 단계의 메모리와 연산을 줄인다.
- attention 이 낮더라도 희귀 이벤트·상태 변화·전이 시작점 같은 구조적 원인 후보를 보호할 수 있다.
- 원인 로그 한 줄뿐 아니라 중간 증거와 결과까지 evidence chain 으로 보존할 수 있다.
- 시퀀스와 카운트를 결합해 사건 순서와 반복 심각도를 함께 제공할 수 있다.
- 같은 메모리 예산에서 일반 압축보다 RCA 에 특화된 정보 배분이 가능하다.

8.2 효과가 특히 클 것으로 예상되는 로그

- Heartbeat·Retry·Polling 처럼 동일 템플릿 반복이 매우 많은 경우
- 수십만 줄 중 한 번 등장한 설정 변경이 원인인 경우
- 템플릿은 같지만 파라미터 변화가 원인인 경우
- 원인과 최종 ERROR 가 문맥에서 멀리 떨어진 경우
- 여러 서비스의 호출·전파 관계가 필요한 멀티홉 장애

8.3 오히려 나빠질 수 있는 위험

위험	실패 예시	대응
이중 정보 손실	전처리에서 한 번, KV 압축에서 다시 한 번 증거 삭제	각 단계별 evidence recall 을 별도로 측정
잘못된 템플릿 병합	서로 다른 상태를 같은 template 로 압축	주요 parameter 상태와 변화 경계 유지
잘못된 장애 구간	원인이 윈도우 밖에 존재	여러 윈도우·식별자·서비스 문맥 비교
식별자 누락·재사용	다른 요청이 합쳐지거나 같은 요청이 갈림	시간 gap 과 서비스 관계를 함께 사용
구조 규칙의 과신	ERROR 가 항상 결과, INFO 가 항상 원인이라는 단정	attention·ablation·과거 사례와 결합
압축 오버헤드	그래프·민감도 계산 비용이 절감량보다 큼	저비용 구조 필터 후 일부 후보만 정밀 평가

기대	현실적인 기대 Full Cache 는 정보 보존의 상한선에 가깝다. 가장 가능성이 높은 성과는 Full Cache 보다 항상 더 정확해지는 것이 아니라, 같은 압축률에서 기존 KV 방법보다 높은 RCA 정확도와 증거 보존율을 얻는 것이다.
----	---

9. 실험 설계와 비교 기준

9.1 필수 비교군

실험	LLM 입력	KV Cache	확인하는 질문
A	원시 로그	Full	전체 시스템의 상한선과 기본 성능
B	원시 로그	기존 KV 압축	일반 압축만 적용한 효과
C	구조화 로그	Full	전처리·구조화 자체의 효과
D	구조화 로그	기존 KV 압축	구조화 입력에서도 기존 압축이 작동하는가
E	구조화 로그	제안 로그 구조 기반 압축	로그 특화 정책의 추가 이득

9.2 공정한 비교 조건

- 모든 KV 압축 방식에 동일한 KV 예산(예: 원본의 10%, 20%, 40%)을 적용한다.
- 동일한 모델·프롬프트·출력 길이·디코딩 설정을 사용한다.
- 장애 사건 단위 held-out 분리 후 각 split 내부에서 윈도우를 생성한다.
- 압축 정책 계산 시간을 포함한 end-to-end latency 를 측정한다.
- 단일 원인, 희귀 원인, 파라미터 원인, lost-in-the-middle, 멀티홉을 분리 평가한다.

9.3 평가 지표

범주	주요 지표
RCA 성능	Root-cause accuracy, Top-k accuracy, 원인 유형·설명 정확성
증거 보존	원인 로그 recall, 원인 파라미터 보존율, evidence chain 보존율
효율	KV 감소율, 최대 GPU 메모리, prefill/decode latency, throughput
전처리 품질	파싱 정확도, 원인 포함 윈도우 비율, 요청 혼입률, 분할 정확도
안정성	압축률별 성능 곡선, 데이터셋·모델 간 일반화, 오버헤드

9.4 가장 빠른 반증 실험

1. 원인 라벨이 있는 소규모 장애 샘플에서 template+parameter 기반 RLE 입력을 만든다.
2. Full Cache, Recent-only, H2O/SnapKV 계열, 제안 마스크를 동일 20% 예산으로 비교한다.
3. 실제 원인 line 의 토큰 KV 가 보존되었는지와 RCA 답변 변화를 함께 측정한다.
4. 제안 방법이 원인 line 을 더 잘 보존하지 못하면 복잡한 head-aware 단계로 확장하지 않는다.

중단

킬 스위치

로그 구조를 이용해도 동일 KV 예산에서 원인 증거 recall 과 RCA 성능이 기존 방법보다 개선되지 않거나, 압축 계산 오버헤드가 절감 효과보다 크면 정책을 단순화하거나 중단한다.

10. 최종 권장 파이프라인과 구현 체크리스트

10.1 권장 end-to-end 흐름



1. 기본 필드 추출: timestamp, level, component, node, identifier, label 을 파싱한다.
2. 장애 사건 정의: 라벨·실패 요청·장애 시각을 anchor 로 만들고 연속 장애를 하나의 사건으로 병합한다.
3. 장애 구간 구성: 식별자 로그를 우선 수집하고, 관련 서비스의 전후 시간 문맥을 추가한다.
4. 정밀 파싱: 템플릿과 파라미터를 분리하고 식별자·핵심 값은 마스킹 전에 별도 저장한다.
5. 특징 생성: RLE 시퀀스, 이벤트 카운트, 파라미터 변화, 전이·서비스 그래프를 만든다.
6. LLM 입력: 전체 통계 + 핵심 증거 시퀀스 + 대표 원문을 계층적으로 제공한다.
7. 토큰-로그 매핑: 각 line/chunk 가 어떤 토큰 범위와 KV 위치에 대응하는지 기록한다.
8. 압축: 동일 상태 반복은 대표화하고 상태·식별자·서비스·전이 경계를 보호한다.
9. 평가: 답변 정확도, 증거 보존, KV 메모리, 전체 latency 를 동시에 측정한다.

10.2 권장 구조화 레코드

구현 데이터 구조 예시

```

LogChunk {
  event_id, template,
  parameters, parameter_change,
  repeat_count, first_ts, last_ts,
  level, service, host,
  trace_id, request_id, job_id,
  predecessor_ids, successor_ids,
  raw_representative_line,
  token_start, token_end
}
  
```

10.3 단계별 체크리스트

단계	확인 항목
파싱	template 과 parameter 를 모두 저장했는가? request/job/block ID 를 숫자로 지우지 않았는가?
장애 구간	원인이 윈도우 안에 포함되는 비율을 측정했는가? 여러 윈도우 크기를 비교했는가?
분할	같은 요청 로그가 묶이고 무관한 요청 혼입이 낮은가? ID 재사용을 시간 gap 으로 분리했는가?
특징	시퀀스·카운트·파라미터·그래프를 각각 ablation 할 수 있는가?
LLM 입력	대표 원문을 남겼는가? 반복 횟수와 상태 변화가 명시되어 있는가?
KV 압축	토큰-라인 매핑이 정확한가? 같은 상태 반복과 상태 변화 경계를 구분하는가?
평가	동일 KV 예산 비교인가? 전처리와 압축 오버헤드를 포함했는가?

정의	프로젝트 정의 문장 본 연구는 긴 운영 로그를 LLM 으로 RCA 할 때, 로그 파싱·장애 구간 추출·식별자 분할·시퀀스/카운트/파라미터/그래프 구조를 이용해 반복 KV 는 압축하고 원인 상태 변화와 evidence chain 에 해당하는 KV 를 보존하는 로그 도메인 특화 KV Cache 압축 전략을 설계한다.
----	--

11. 핵심 용어와 참고문헌

11.1 핵심 용어

용어	정확한 의미
Parsing	로그 한 줄을 필드·템플릿·파라미터로 구조화하는 작업
Failure window extraction	장애 anchor 주변에서 관련 로그 범위를 선택하는 작업
Partitioning	같은 요청·작업·서비스에 속한 로그를 그룹화하는 작업
Event sequence	이벤트의 발생 순서와 실행 흐름을 표현한 특징
Event count	윈도우 내 이벤트별 발생 횟수 특징
Graph feature	서비스·요청·로그 사이 호출·전이·소속 관계 표현
Root cause	사용자 실패를 일으킨 근본 결함·설정·상태 변화
Evidence chain	원인 후보→중간 전파 증거→최종 실패의 연결

용어	정확한 의미
KV Cache	디코딩 시 과거 토큰의 Key·Value 를 재사용하기 위해 저장하는 상태
KV eviction	KV 예산을 줄이기 위해 일부 위치를 제거하거나 대표화하는 정책

11.2 핵심 결론

결론	최종 결론 이 연구의 경쟁력은 로그를 단순히 짧게 만드는 데 있지 않다. 로그 구조를 이용해 무엇이 반복이고 무엇이 상태 변화이며 어떤 로그들이 같은 장애 chain 을 구성하는지 판단하고, 그 판단을 KV 보존 정책까지 연결하는 데 있다.
----	--

11.3 참고문헌

[1] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, Michael R. Lyu. “A Survey on Automated Log Analysis for Reliability Engineering.” ACM Computing Surveys, 2021.

[2] OpenTelemetry. “Logs Data Model.” Status: Stable. Timestamp, TraceId, SpanId, Severity, Body, Attributes, EventName 등의 공통 필드 정의.

[3] LogPAL. “Drain3: A Robust Streaming Log Template Miner Based on the Drain Algorithm.” 공식 GitHub 저장소.

[4] Zhenyu Zhang et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.” arXiv:2306.14048, 2023.

[5] Yuhong Li et al. “SnapKV: LLM Knows What You Are Looking for Before Generation.” arXiv:2404.14469, 2024.

[6] Xiang Liu et al. “ChunkKV: Semantic-Preserving KV Cache Compression for Efficient Long-Context LLM Inference.” arXiv:2502.00299, 2025.

[7] Xiaolin Lin et al. “CompressKV: Semantic-Retrieval-Guided KV-Cache Compression for Resource-Efficient Long-Context LLM Inference.” arXiv:2606.24467, 2026.

— 문서 끝 —